

LAB SESSION 10

Lab Name: Maintenance, Ethics & Final Review

Project: MediScript-E — Digital Healthcare Platform

Objectives

- Understand post-deployment software challenges for MediScript-E
- Address ethical responsibilities in healthcare software development
- Present final project review with complete SDLC summary

Theory

Most software cost occurs during maintenance. After deployment, software must be updated, patched, and improved continuously. Engineers must act ethically, protecting users and data — especially in healthcare systems where patient safety and privacy are paramount.

Task 1: Maintenance Plan

1.1 Types of Maintenance

Type	Description	Examples in MediScript-E
Corrective	Fix bugs and errors discovered after deployment	Fix SSL connection issues, fix OTP expiry bugs, fix duplicate export errors
Adaptive	Adapt to changes in environment or dependencies	Prisma 7 migration, Next.js version upgrades, Groq model updates
Perfective	Improve performance and add new features	Add AI chatbot, global search, prescription archive, profile pictures, dedicated routes
Preventive	Prevent future failures through refactoring	Connection pooling, error handling improvements, ownership authorization

1.2 Scheduled Maintenance Activities

Activity	Frequency	Responsible	Description
Dependency updates	Monthly	Developer	Update npm packages, check for security patches
Security audit	Monthly	Developer	Run <code>npm audit</code> , fix vulnerabilities

Activity	Frequency	Responsible	Description
Database backup verification	Weekly	Supabase (Auto)	Verify Supabase automatic backups are working
Production log review	Weekly	Developer	Review Vercel logs for errors and anomalies
Environment variable rotation	Quarterly	Developer	Rotate NEXTAUTH_SECRET, CRON_API_KEY, GROQ_API_KEY
OAuth credential review	Quarterly	Developer	Verify Google and GitHub OAuth apps are active
Groq API key review	Monthly	Developer	Verify Groq API key is valid and within quota
Performance monitoring	Monthly	Developer	Check Vercel analytics for slow API routes
Dead code cleanup	Quarterly	Developer	Remove unused components and API routes

1.3 Version Control & Release Strategy

Strategy	Implementation
Branch	Single main branch for production
Deployment	Manual vercel --prod CLI for production deployments
Rollback	Vercel dashboard allows instant rollback to previous deployment
Hotfix	Direct commit to main for critical production fixes
Feature development	Local development → pnpm run build → commit → push → deploy

1.4 Known Technical Debt & Future Improvements

Item	Priority	Description
Rate limiting on API routes	High	Prevent brute force and DDoS attacks on login and OTP endpoints
Automated test suite (Jest)	High	Replace manual testing with automated unit and integration tests
Email queue system	Medium	Use a proper email queue instead of direct SMTP calls
Refresh token rotation	Medium	Implement refresh token rotation for enhanced session security
Doctor availability calendar	Medium	Add time slot management for doctors
Real-time notifications	Medium	WebSockets or Server-Sent Events for live updates
Audit logging	Medium	Log all admin actions for accountability

Item	Priority	Description
Custom profile picture upload	Medium	Allow email/password users to upload profile pictures
TOTP-based 2FA	Low	Add Google Authenticator support as alternative to email OTP
Multi-language support	Low	Add Bengali language support for local users
GDPR compliance	Medium	Add account deletion and data export features
Mobile app	Low	React Native app for patients and doctors
Video consultations	Low	Premium feature for future implementation

1.5 Monitoring & Alerting

Tool	Purpose	Current Status
Vercel Analytics	Page load performance, visitor tracking	Active
Vercel Logs	Runtime errors, API response times	Active
Supabase Dashboard	Database query performance, storage usage	Active
GitHub	Code changes, commit history	Active
Manual testing	Feature verification after each deployment	Active

Task 2: Ethics Analysis

2.1 Ethical Responsibilities in Healthcare Software

As developers of MediScript-E, we have ethical responsibilities to:

- **Protect patient privacy:** Medical data is highly sensitive and must be secured
- **Ensure system reliability:** Downtime or bugs can affect patient care
- **Prevent unauthorized access:** Only authorized users should access medical records
- **Be transparent:** Users should know how their data is used
- **Avoid harm:** System errors must not lead to incorrect medical decisions
- **Responsible AI:** AI chatbot must not provide harmful medical advice

2.2 Ethical Issues Identified

Issue ID	Ethical Issue	Category	Impact	Mitigation
E-01	Patient medical data stored in cloud (Supabase)	Privacy	High	SSL encryption, access control, Supabase security policies
E-02	Email addresses used for OTP and reminders	Privacy	Medium	Emails used only for system notifications, never shared

Issue ID	Ethical Issue	Category	Impact	Mitigation
E-03	Admin can view and delete any user account	Power misuse	High	Self-deletion prevented, admin actions visible in logs
E-04	OAuth login auto-creates patient accounts	Consent	Medium	Users explicitly click OAuth button — implied consent
E-05	Prescription data accessible to both doctor and patient	Data sharing	Medium	RBAC ensures only relevant parties access prescriptions
E-06	Medicine reminder emails sent automatically	Autonomy	Low	Users set reminders themselves, can delete anytime
E-07	System downtime could delay medical access	Reliability	High	99.9% uptime target, Vercel auto-scaling
E-08	Fake doctor registration with invalid license	Fraud	Critical	License number uniqueness enforced, manual verification recommended
E-09	Patient blood group stored without explicit consent notice	Informed consent	Medium	Blood group is optional profile data, user selects during registration
E-10	No data deletion/export feature for users (GDPR)	User rights	Medium	Future improvement: add account deletion and data export
E-11	AI chatbot could provide incorrect platform information	Misinformation	Medium	System prompt carefully designed, responses reviewed
E-12	AI chatbot could be used to extract system information	Security	Medium	System prompt instructs MediBot to never reveal internal details
E-13	OAuth profile pictures stored from external providers	Privacy	Low	Images loaded directly from provider URLs, not stored locally

2.3 Privacy & Data Protection Measures

Measure	Implementation
Data minimization	Only necessary data collected (no unnecessary PII)
Password protection	bcryptjs hashing — passwords never stored in plaintext
Secure transmission	HTTPS enforced on all routes via Vercel
Database encryption	SSL/TLS connection to Supabase PostgreSQL

Measure	Implementation
Access control	RBAC — users only access their own data
Token expiry	Verification tokens (24h), OTP (10min), sessions (30 days)
Environment secrets	All credentials in <code>.env</code> — never in source code
File security	Medical documents stored in Supabase with access control
AI safety	MediBot system prompt prevents harmful medical advice
Search privacy	Search results filtered by user role and ownership

2.4 ACM Code of Ethics — Applied to MediScript-E

ACM Principle	Application in MediScript-E
Contribute to society and human well-being	MediScript-E improves healthcare access for patients and doctors
Avoid harm	System tested with 82 test cases; AI chatbot restricted from medical advice
Be honest and trustworthy	No fake features — all documented features are implemented and accurate
Respect privacy	Patient medical data protected with encryption and RBAC
Honor confidentiality	Medical records accessible only to authorized users
Give comprehensive and thorough evaluations	All modules tested before deployment

Task 3: Final Project Report

3.1 Project Summary

Field	Details
Project Name	MediScript-E
Type	Digital Healthcare Platform
Live URL	https://mediscript-e.vercel.app
Repository	https://github.com/tusharsno/mediscript-e
Developer	Tushar
University	USTC, Chittagong
Tech Stack	Next.js 16, React 19, TypeScript, Prisma 7, PostgreSQL, Supabase, NextAuth, Groq AI, Vercel

3.2 SDLC Summary

Phase	Lab Session	Deliverable	Status
Requirements Engineering	Lab 3	27 FR, 18 NFR, Priority Matrix	 Complete
SRS Documentation	Lab 4	IEEE-standard SRS with 59 FRs across 7 modules	 Complete
Use Case Modeling	Lab 5	9 Actors, 15 Use Case Specifications	 Complete
System Design	Lab 6	Architecture, DB Schema, Route Design, UML Diagrams	 Complete
Security Engineering	Lab 7	18 Failure Scenarios, 17 Threats, Security Controls, RBAC Matrix	 Complete
Testing	Lab 8	Test Plan, 82 Test Cases, RTM	 Complete
Implementation	Lab 9	Source Code, 10 Modules, Git Repository	 Complete
Maintenance & Ethics	Lab 10	Maintenance Plan, Ethics Analysis, Final Report	 Complete

3.3 Features Implemented

Feature	Status	Notes
User Registration (Patient/Doctor)		With blood group and license number
Email Verification		24-hour token expiry
Credential Login		bcryptjs password hashing
Google OAuth Login		Auto-creates patient account + profile picture
GitHub OAuth Login		Auto-creates patient account + profile picture
Two-Factor Authentication (2FA)		Email OTP, 10-minute expiry
Appointment Booking		Patient books with doctor, date, time
Appointment Management		Doctor confirms/cancels/completes with confirmation dialogs
Digital Prescriptions		Doctor issues, patient views
Prescription Archive/Edit/Delete		Doctor manages prescription lifecycle

Feature	Status	Notes
Auto-Complete Appointment	✓	Appointment auto-completed when prescription issued
PDF Download	✓	html2canvas + jsPDF
Medicine Reminders	✓	With automated email alerts via cron
Medical Vault	✓	Supabase Storage file upload
Admin Dashboard	✓	Real-time stats, user management
Settings (Profile/Password/2FA)	✓	Full account management
Contact Form	✓	Public form with admin view
AI Chatbot (MediBot)	✓	Groq Llama 3.1, English only, plain text
Global Search	✓	Role-based, debounced, dropdown results
OAuth Profile Pictures	✓	Google/GitHub photos in navbar and sidebar
Dedicated Feature Pages	✓	Professional route structure
Page Transition Animations	✓	Framer Motion fade + slide
Dashboard Search Bar	✓	In header with 350ms debounce
Responsive Design	✓	Mobile, tablet, desktop
Production Deployment	✓	Vercel with live URL

3.4 Project Statistics

Metric	Value
Total API Endpoints	40+
Database Models	9
React Components	30+
Functional Requirements	59
Non-Functional Requirements	25
Use Cases	15
Test Cases	82
Security Threats Identified	17
Failure Scenarios Identified	18
Page Routes	15+
Git Commits	20+
Lines of Code (approx.)	8000+

3.5 Challenges & Solutions

Challenge	Solution
Prisma 7 breaking changes on Vercel	Moved connection URLs to <code>prisma.config.ts</code> , used <code>@prisma/adapters-pg</code>
SSL/TLS certificate error on Supabase	Added <code>sslmode=no-verify</code> to <code>DATABASE_URL</code>
Vercel deploying old commit	Used <code>vercel --prod</code> CLI to bypass GitHub webhook issue
2FA bypass security vulnerability	Added <code>twoFactorCode !== null</code> check before session creation
Groq API quota exceeded	Switched to new API key with fresh quota
Gemini API not working	Switched from Gemini to Groq for AI chatbot
AI responses with markdown formatting	Added regex-based markdown stripping in API route
Duplicate export error in components	Rewrote files cleanly to remove duplicate function definitions
Hash-based navigation not highlighting sidebar	Migrated to dedicated routes for proper pathname-based highlighting
OAuth profile pictures blocked by Next.js	Added Google and GitHub image domains to <code>next.config.ts</code>
<code>package-lock.json</code> conflict with pnpm	Deleted <code>package-lock.json</code> , kept only <code>pnpm-lock.yaml</code>

3.6 Lessons Learned

- **Professional structure matters:** Dedicated routes per feature is the industry standard, not single-page scroll navigation
- **Requirements first:** Clear requirements in Lab 3 & 4 prevented scope creep during implementation
- **Security by design:** Implementing RBAC and 2FA from the start is easier than adding later
- **Version control is critical:** Git allowed safe rollback when Vercel deployment broke
- **Production ≠ Development:** SSL, environment variables, and serverless constraints behave differently in production
- **AI integration requires care:** System prompt design is critical for safe and useful AI responses
- **User experience details matter:** Profile pictures, page transitions, and search bars significantly improve professional feel
- **Test early:** Security test cases revealed RBAC gaps that were fixed before final deployment

Key Findings / Learning Outcomes

- Understood that **maintenance is the longest and most expensive phase** of the software lifecycle
- Learned that healthcare software carries **higher ethical responsibility** due to sensitive medical data

- Recognized that **GDPR compliance** (data deletion, export) is a future requirement for MediScript-E
- Understood that **fake doctor registration** is a critical ethical risk requiring manual verification
- Completed a full **SDLC cycle** from requirements engineering to deployment and maintenance planning
- Recognized that **AI integration** introduces new ethical considerations around content safety and misinformation
- Learned that **professional web app structure** (dedicated routes, search, profile pictures, animations) significantly improves user experience and credibility